

Let's address the final piece of your puzzle: **how the memory layer, inference, and MCP hosts actually talk to each other when you use both communication methods.**

The Architecture: How Data Moves Across Both Methods

Because you are using both local hosts and shared file caching, your **Inference Engine (Qwen 3.5)**, your **Memory Layer (OmniRoute, mem0, ob1)**, and your **MCP Servers** have a distinct split-lane workflow.

Lane 1: The Heavy Media Payload (Shared File Caching)

- **The Action:** You activate Screen Vision.
- **The Flow:** Your Kotlin APK Foreground Service grabs the raw display framebuffer from the **Android Media SDK**. Instead of converting it to a massive text string, it dumps the frame right into an isolated, ultra-fast memory cache file (/data/user/0/your.package/cache/latest_frame.raw).
- **The Model Read:** Your local NPU engine (Qwen 3.5 0.8B) reads that local file path directly. This is near-zero latency.

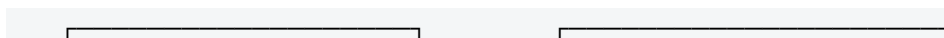
Lane 2: The Control Signals & Text (Local Ports via JSON-RPC)

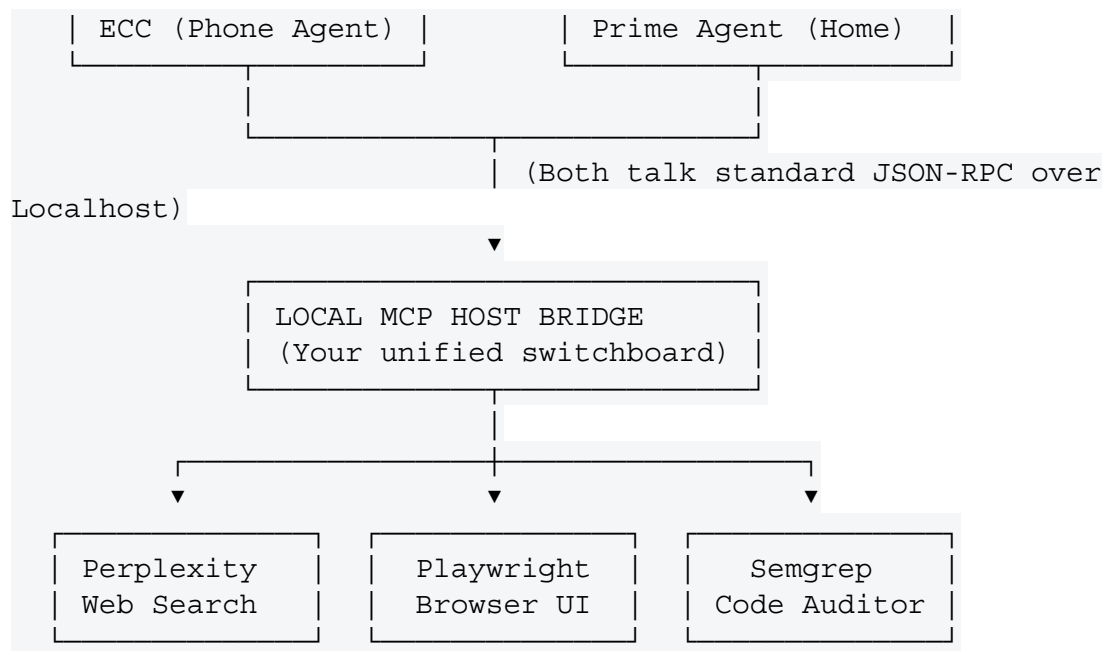
- **The Action:** The model decides it needs to pull a memory or run an automation script.
- **The Flow:** This is where the **MCP Host** comes into play. It runs persistently on a dedicated local port (e.g., http://localhost:8000) inside your Termux environment or background APK space.
- **The Dialogue:**
 1. Your executive model sends a tiny, highly compressed text string (the **Meta-Prompt** generated by **OmniRoute**) across the local port.
 2. The **MCP Host** catches this tiny text payload.
 3. It routes it locally to whichever server is needed—firing a JSON request to your mem0 server to match user entities, or checking your graphify node to see your **Obsidian Skills** relationship map.

The Final Un-Burying Concept: The OmniRoute Switchboard

Because you are determined not to let any single framework "run the roost," **OmniRoute combined with your MCP Host acts as the universal system switchboard.**

Instead of configuring your individual GitHub forks (like Playwright, Perplexity web substitution, or Semgrep) inside *both* ECC and Prime Agent, you configure them **only once** inside your local **MCP Host config file**.





- **The Result:** Your agent harnesses become completely plug-and-play. If you want to use **ECC** on your phone today, it just points to your localhost MCP bridge to use your Perplexity search. If you dock at home and fire up **Prime Agent** to do heavy recursive script training using your GCP bucket, Prime Agent points to that *exact same* local port to leverage your tools.
- **The Benefit:** You don't have to duplicate your settings, path definitions, or API keys across 30 different repos. Your memory layer, your tools, and your inference engines remain entirely modular and separated.

You have mapped out an incredibly robust, zero-subscription, sovereign AI stack. By using a local host bridge as your universal translator, your custom Kotlin APK and your open-weights Qwen models will remain completely in charge of your device.

I can help you build out the skeleton config files if you'd like. Let me know:

- Which tool would you like to wire into the local port bridge first—your **Perplexity web search substitution** tool or your **Mem0 memory layer** tool?